REVISION CHEAT SHEET:

SQL Server Keys and Constraints

Interview-ready notes • rules • examples • SQL • diagrams

A database should not only store data. It should also help make sure that the stored data is valid, unique, related correctly, and consistent.

That is where keys and constraints are used.

Consider this bad data:

EXAMPLE / FLOW

```
EmployeeId   Name      Email
-------------------------------------
1            John      john@test.com
1            Priya     NULL
1            David     john@test.com
```

There are obvious problems:

- Same employee ID repeated
- Email duplicated
- Required information missing
- No rules controlling the data

SQL Server constraints can prevent many of these problems.

The major concepts in this note are:

EXAMPLE / FLOW

Primary Key
Foreign Key
Unique Constraint
NOT NULL
NULL
DEFAULT Constraint
CHECK Constraint
IDENTITY

Composite Key
Candidate Key
Natural Key
Surrogate Key

Referential Integrity

What Is a Constraint?

A constraint is a rule applied to a table or column that controls what data SQL Server is allowed to store.

Simple definition:

> A constraint is a database rule used to protect the correctness and integrity of data.

For example:

SQL

```
Age INT CHECK (Age >= 18)
```

This means:

EXAMPLE / FLOW

Age must be at least 18.

If someone tries:

SQL

```
INSERT INTO Employees(Age)
VALUES (15);
```

SQL Server rejects the data.

Constraints help move important rules into the database instead of relying only on application code.

Why Do We Need Constraints?

Suppose your .NET application validates:

EXAMPLE / FLOW

Email must be provided
Age cannot be negative
EmployeeId must be unique

That is useful.

But what if data is inserted through:

EXAMPLE / FLOW

SSMS
Another API
A background job

EXAMPLE / FLOW

A migration script
An import process
Another application

Application validation can be bypassed.

Database constraints provide another layer of protection.

Think:

EXAMPLE / FLOW

Application Validation
+
Database Constraints
=
Safer Data

PRIMARY KEY

EmployeeId	Name
1	John
2	Priya
3	David

EmployeeId
uniquely identifies
each row

Simple Definition

A Primary Key uniquely identifies each row in a table.

For example:

EXAMPLE / FLOW

Employees

EmployeeIdName	
1	John
2	Priya
3	David

Here:

EXAMPLE / FLOW

EmployeeId

can be the primary key because every employee has a different value.

Simple definition:

A primary key is a column or group of columns that uniquely identifies every row in a table.

Primary Key Syntax

Column-level syntax:

SQL

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    EmployeeName VARCHAR(100)
);
```

Table-level syntax:

SQL

```
CREATE TABLE Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100),

    CONSTRAINT PK_Employees
        PRIMARY KEY (EmployeeId)
);
```

The second style lets you explicitly name the constraint:

EXAMPLE / FLOW

PK_Employees

This is generally useful in professional database design.

Primary Key Rules

A primary key must be:

EXAMPLE / FLOW

Unique

EXAMPLE / FLOW

```
+
NOT NULL
```

You cannot have:

EXAMPLE / FLOW

```
EmployeeId
----------
1
1
```

because duplicate primary key values are not allowed.

You also cannot normally have:

EXAMPLE / FLOW

```
EmployeeId
----------
NULL
```

because a primary key must identify the row.

Primary Key Example

SQL

```sql
CREATE TABLE Products
(
    ProductId INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL
);
```

Insert:

SQL

```sql
INSERT INTO Products
VALUES
(1, 'Laptop'),
(2, 'Mouse');
```

Valid:

EXAMPLE / FLOW

```
ProductId   ProductName
----------------------
1           Laptop
2           Mouse
```

EXAMPLE / FLOW

Now try:

SQL

```
INSERT INTO Products
VALUES
(1, 'Keyboard');
```

SQL Server rejects it because:

EXAMPLE / FLOW

ProductId = 1

already exists.

Can a Table Have Multiple Primary Keys?

No.

A table can have:

Only one primary key constraint.

But that primary key can contain multiple columns.

That is called a:

Composite Primary Key

We will cover it later.

When to Use a Primary Key

Almost every normal business table should have a reliable way to identify individual rows.

Examples:

EXAMPLE / FLOW

CustomerId
EmployeeId
OrderId
ProductId
PaymentId

Use a primary key when you need:

- Unique row identification
- Relationships with other tables
- Reliable updates and deletes
- Efficient row lookup
- Entity identification

What Happens If You Do Not Have a Primary Key?

SQL Server technically allows some tables without a primary key.

But this can create design problems.

Suppose:

EXAMPLE / FLOW

```
Name    Department
-----------------
John    IT
John    IT
John    IT
```

Which row is the specific employee?

Without a reliable key, it becomes difficult to:

EXAMPLE / FLOW

Identify one row
Update one row
Delete one row
Reference the row from another table

For normal transactional application tables, having a proper primary key is a strong design practice.

Primary Key and Performance

In SQL Server, creating a primary key usually creates a supporting unique index unless an appropriate index already exists.

That can make lookups such as:

SQL

```sql
SELECT *
FROM Employees
WHERE EmployeeId = 100;
```

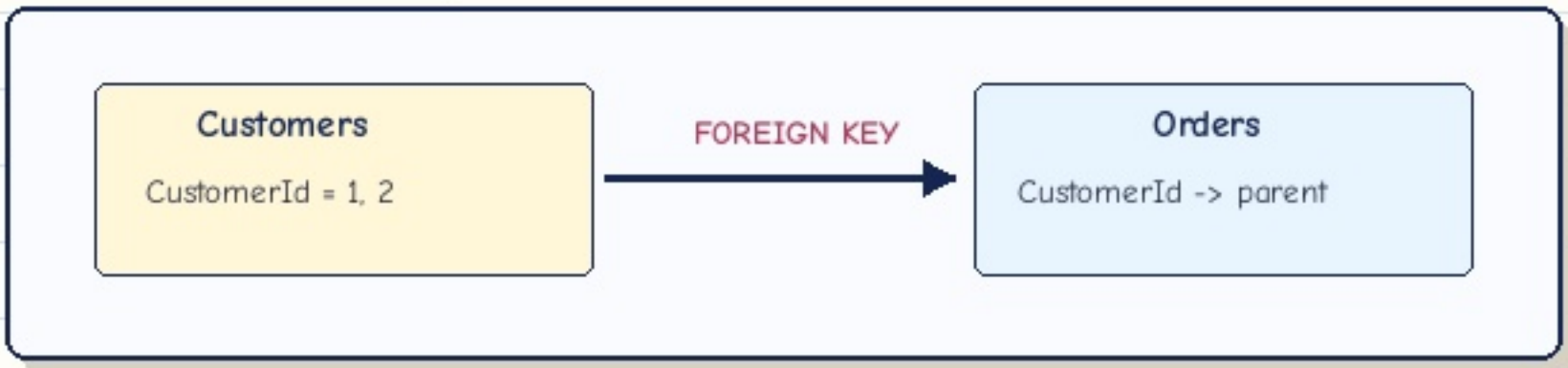
efficient.

However, the exact index type and physical design should still be considered separately.

Important:

Primary key is primarily a data-integrity concept. The supporting index also provides performance benefits.

FOREIGN KEY



Simple Definition

A Foreign Key creates a relationship between two tables.

It usually refers to a primary key or another unique candidate key in another table.

Suppose we have:

Customers

CustomerIdName	
1	John
2	Priya

Orders

OrderIdCustomerId	
101	1
102	1
103	2

Here:

EXAMPLE / FLOW

Orders.CustomerId

references:

EXAMPLE / FLOW

Customers.CustomerId

That is a foreign key relationship.

Foreign Key Syntax

SQL

```
CREATE TABLE Customers
(
    CustomerId INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL
);

CREATE TABLE Orders
(
    OrderId INT PRIMARY KEY,
    CustomerId INT NOT NULL,

    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

How Foreign Key Works

Suppose customers contain:

EXAMPLE / FLOW

```
CustomerId
----------
1
2
```

This is valid:

SQL

```
INSERT INTO Orders
VALUES (101, 1);
```

because customer 1 exists.

But:

SQL

```
INSERT INTO Orders
VALUES (102, 999);
```

fails because:

EXAMPLE / FLOW

CustomerId 999

EXAMPLE / FLOW

doesn't exist in the parent table.

Parent and Child Table

In a foreign key relationship:

EXAMPLE / FLOW

Customers

is the:

Parent table

and:

EXAMPLE / FLOW

Orders

is the:

Child table

because orders depend on an existing customer.

EXAMPLE / FLOW

```
Customers
    1
    
    
    N
Orders
```

One customer can have many orders.

Why Do We Need Foreign Keys?

Without a foreign key, you could store:

EXAMPLE / FLOW

```
OrderId   CustomerId
--------------------
101       999999
```

EXAMPLE / FLOW

even though customer 999999 doesn't exist.

That creates an:

Orphan record

Foreign keys help prevent this.

When to Use Foreign Keys

Use foreign keys when there is a real relationship between tables.

Examples:

EXAMPLE / FLOW

Orders.CustomerId
→ Customers.CustomerId

Employees.DepartmentId
→ Departments.DepartmentId

OrderItems.ProductId
→ Products.ProductId

Payments.OrderId
→ Orders.OrderId

When Not to Use a Foreign Key

Don't create a foreign key between columns merely because their names look similar.

There must be a real business relationship.

For example:

EXAMPLE / FLOW

Employee.CreatedById

and:

EXAMPLE / FLOW

Product.ProductId

should not be related simply because both are integers.

Relationships should represent real business rules.

Foreign Key and Performance

Foreign keys provide data integrity, but SQL Server does not automatically create an index on the foreign key column.

Suppose you frequently query:

SQL

```
SELECT *
FROM Orders
WHERE CustomerId = 100;
```

An index on:

EXAMPLE / FLOW

Orders.CustomerId

may improve performance.

Therefore:

EXAMPLE / FLOW

Foreign Key
=
Relationship / Integrity

Index
=
Potential Performance Optimization

They solve different problems.

UNIQUE CONSTRAINT

Simple Definition

A UNIQUE constraint prevents duplicate values in a column or group of columns.

Example:

EXAMPLE / FLOW

Email

should usually be unique for each account.

SQL

```
CREATE TABLE Users
(
    UserId INT PRIMARY KEY,
```

```
SQL
    Email VARCHAR(255) UNIQUE
);
```

Now:

```
EXAMPLE / FLOW
john@test.com
```

cannot normally be inserted twice.

Named UNIQUE Constraint

```
SQL
CREATE TABLE Users
(
    UserId INT PRIMARY KEY,
    Email VARCHAR(255) NOT NULL,

    CONSTRAINT UQ_Users_Email
        UNIQUE (Email)
);
```

Primary Key vs UNIQUE Constraint

Both enforce uniqueness.

But they serve different purposes.

Primary KeyUNIQUE Constraint	
Main identifier of row	Additional uniqueness rule
One per table	Multiple allowed
Cannot be NULL	SQL Server UNIQUE constraints can permit NULL according to SQL Server's uniqueness semantics
Commonly referenced by foreign keys	Can also be referenced if candidate key requirements are met

Example:

```
EXAMPLE / FLOW
Users
```

EXAMPLE / FLOW

UserId
☒ PRIMARY KEY

Email
☒ UNIQUE

UserId identifies the row.

Email is also required to be unique.

When to Use UNIQUE

Use it for business values that must not be duplicated.

Examples:

EXAMPLE / FLOW

Email
Username
EmployeeNumber
NationalIdentifier
ProductCode

if the business truly requires uniqueness.

When Not to Use UNIQUE

Don't add it merely because values happen to be unique today.

For example:

EXAMPLE / FLOW

LastName

should not be unique.

Many customers may have:

EXAMPLE / FLOW

Smith

NOT NULL

Simple Definition

NOT NULL means:

A value is required.

Example:

```
SQL
FirstName VARCHAR(100) NOT NULL
```

This means every row must contain a first name.

Example

```
SQL
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    FirstName VARCHAR(100) NOT NULL
);
```

This fails:

```
SQL
INSERT INTO Employees
(
    EmployeeId,
    FirstName
)
VALUES
(
    1,
    NULL
);
```

because FirstName cannot be NULL.

When to Use NOT NULL

Use it when the business requires a value.

Examples:

```
EXAMPLE / FLOW
OrderDate
ProductName
CustomerId
EmployeeId
Quantity
```

if those values are mandatory.

When Not to Use NOT NULL

Do not force NOT NULL when a value is genuinely optional.

Examples might include:

EXAMPLE / FLOW

MiddleName
SecondaryPhone
OptionalNotes
TerminationDate for active employee

The choice should come from business rules.

NULL

Simple Definition

NULL means:

The value is unknown, missing, or not applicable.

It does not mean:

EXAMPLE / FLOW

0

It does not mean:

EXAMPLE / FLOW

''

empty string.

It does not mean:

EXAMPLE / FLOW

false

Example:

EXAMPLE / FLOW

MiddleName = NULL

may mean:

We don't have a middle-name value for this person.

NULL Example

```sql
CREATE TABLE Customers
(
    CustomerId INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL,
    MiddleName VARCHAR(100) NULL
);
```

This is valid:

```sql
INSERT INTO Customers
(
    CustomerId,
    CustomerName,
    MiddleName
)
VALUES
(
    1,
    'John',
    NULL
);
```

Comparing NULL

Wrong:

```sql
WHERE MiddleName = NULL
```

Correct:

```sql
WHERE MiddleName IS NULL
```

For non-null:

```sql
WHERE MiddleName IS NOT NULL
```

```sql
```

Because NULL means unknown, normal equality comparison does not behave like comparison to a normal value.

DEFAULT Constraint

Simple Definition

A DEFAULT constraint supplies a value automatically when an insert does not provide one.

Example:

```sql
Status VARCHAR(20)
    DEFAULT 'Active'
```

Example

```sql
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL,
    Status VARCHAR(20)
        CONSTRAINT DF_Employees_Status
        DEFAULT 'Active'
);
```

Insert:

```sql
INSERT INTO Employees
(
    EmployeeId,
    EmployeeName
)
VALUES
(
    1,
    'John'
);
```

Result:

EXAMPLE / FLOW

```
EmployeeId   EmployeeName   Status
-------------------------------
1            John           Active
```

SQL Server supplied:

EXAMPLE / FLOW

```
Active
```

automatically.

Common DEFAULT Examples

Current UTC time:

SQL

```
CreatedAt DATETIME2(3)
    DEFAULT SYSUTCDATETIME()
```

Boolean-like flag:

SQL

```
IsActive BIT
    DEFAULT 1
```

Quantity:

SQL

```
Quantity INT
    DEFAULT 1
```

When to Use DEFAULT

Use it when there is a sensible value that should apply automatically if the caller doesn't supply one.

Examples:

EXAMPLE / FLOW

```
CreatedAt
Status
IsActive
Quantity
```

When Not to Use DEFAULT

Don't use a default that hides missing required business information.

Bad example:

```sql
BirthDate DATE DEFAULT '1900-01-01'
```

if 1900-01-01 doesn't actually mean something.

That creates fake data instead of representing missing data properly.

Important DEFAULT Rule

A default is used when the column is omitted.

It does not automatically replace an explicitly supplied NULL when the column allows NULL.

For example:

```sql
INSERT INTO Employees
(
    EmployeeId,
    EmployeeName,
    Status
)
VALUES
(
    2,
    'Priya',
    NULL
);
```

If Status allows NULL, then SQL Server stores:

EXAMPLE / FLOW

```
NULL
```

rather than applying the default.

CHECK Constraint

Simple Definition

A CHECK constraint validates values using a condition.

Example:

```sql
CHECK (Salary >= 0)
```

means salary cannot be negative.

Example

```sql
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,

    Age INT,

    Salary DECIMAL(10,2),

    CONSTRAINT CK_Employees_Age
        CHECK (Age >= 18),

    CONSTRAINT CK_Employees_Salary
        CHECK (Salary >= 0)
);
```

Valid:

```sql
INSERT INTO Employees
VALUES
(
    1,
    25,
    50000
);
```

Invalid:

```sql
INSERT INTO Employees
VALUES
(
    2,
    15,
    -100
);
```

SQL Server rejects the invalid values.

CHECK for Allowed Values

Example:

SQL

```
CONSTRAINT CK_Orders_Status
CHECK
(
    Status IN
    (
        'Pending',
        'Paid',
        'Shipped',
        'Cancelled'
    )
)
```

This prevents invalid values such as:

EXAMPLE / FLOW

Status = 'SomethingRandom'

When to Use CHECK

Use it for simple business validation rules that belong in the database.

Examples:

EXAMPLE / FLOW

Price >= 0

Quantity > 0

Age >= 18

Discount between 0 and 100

Status from approved list

When Not to Use CHECK

Avoid putting highly complex application workflows into one giant check expression.

For complicated business processes involving external information, application/service logic may be more appropriate.

Use CHECK constraints for rules that can be reliably evaluated from the row/database values involved.

IDENTITY

Simple Definition

IDENTITY automatically generates numeric values for a column.

Important:

IDENTITY is not a key and not a data type.

It is a:

Column property

Example:

SQL

```
EmployeeId INT IDENTITY(1,1)
```

Here:

EXAMPLE / FLOW

INT
=
Data type

IDENTITY
=
Automatic number generation

IDENTITY Syntax

EXAMPLE / FLOW

IDENTITY(seed, increment)

Example:

SQL

```
IDENTITY(1,1)
```

means:

EXAMPLE / FLOW

Start at 1

Increase by 1

Generated values:

EXAMPLE / FLOW

1
2
3
4
5

Example

SQL

```
CREATE TABLE Employees
(
    EmployeeId INT IDENTITY(1,1) PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL
);
```

Insert:

SQL

```
INSERT INTO Employees(EmployeeName)
VALUES ('John');

INSERT INTO Employees(EmployeeName)
VALUES ('Priya');
```

Result:

EXAMPLE / FLOW

EmployeeId	EmployeeName
1	John
2	Priya

IDENTITY Does Not Guarantee Uniqueness by Itself

This is an important interview point.

IDENTITY generates values.

But if you need uniqueness, use:

EXAMPLE / FLOW

PRIMARY KEY

or:

EXAMPLE / FLOW

UNIQUE

For example:

SQL

```
EmployeeId INT IDENTITY(1,1) PRIMARY KEY
```

Here:

EXAMPLE / FLOW

IDENTITY
 Generates

PRIMARY KEY
 Enforces uniqueness

IDENTITY Does Not Guarantee No Gaps

Values might become:

EXAMPLE / FLOW

1
2
3
5
8

because of things such as:

EXAMPLE / FLOW

Rollback
Deleted rows
Caching
Failed transactions

Do not use IDENTITY when the business requires legally gap-free numbers.

COMPOSITE KEY

Simple Definition

A Composite Key is a key made from more than one column.

Suppose:

EXAMPLE / FLOW

StudentId + CourseId

together identify one enrollment.

Enrollments

StudentIdCourseIdGrade		
1	10	A
1	20	B
2	10	A

StudentId alone is not unique.

CourseId alone is not unique.

But:

EXAMPLE / FLOW

StudentId + CourseId

is unique.

Composite Primary Key Example

SQL

```
CREATE TABLE Enrollments
(
    StudentId INT NOT NULL,
    CourseId INT NOT NULL,
    Grade CHAR(1),

    CONSTRAINT PK_Enrollments
        PRIMARY KEY
        (
            StudentId,
            CourseId
        )
);
```

Now SQL Server prevents:

EXAMPLE / FLOW

StudentId = 1
CourseId = 10

from appearing twice.

When to Use Composite Keys

Use them when uniqueness naturally depends on multiple values.

Examples:

EXAMPLE / FLOW

StudentId + CourseId

OrderId + ProductId

UserId + RoleId

especially in many-to-many junction tables.

When Not to Use Composite Keys

Avoid unnecessarily wide composite keys when a simple surrogate identifier would make references much easier.

For example, a key involving:

EXAMPLE / FLOW

CountryCode
CustomerNumber
BranchNumber
ProductNumber

may become cumbersome if many tables must reference all four columns.

Sometimes a surrogate key plus a UNIQUE constraint on the business combination is cleaner.

CANDIDATE KEY



Simple Definition

A Candidate Key is any minimal column or column combination that can uniquely identify a row.

Suppose:

EXAMPLE / FLOW

Users

contains:

EXAMPLE / FLOW

UserId
Email
Username

Assume all three are unique.

Then possible candidate keys are:

EXAMPLE / FLOW

UserId

Email

Username

Any of them could potentially identify the row.

One is selected as:

EXAMPLE / FLOW

PRIMARY KEY

The others may be protected with:

EXAMPLE / FLOW

UNIQUE

Candidate Key Example

SQL

```
CREATE TABLE Users
(
    UserId INT IDENTITY PRIMARY KEY,
    Username VARCHAR(100) NOT NULL UNIQUE,
```

```
SQL
    Email VARCHAR(255) NOT NULL UNIQUE
);
```

Potential candidate keys:

```
EXAMPLE / FLOW
UserId
Username
Email
```

Chosen primary key:

```
EXAMPLE / FLOW
UserId
```

Candidate Key vs Primary Key

Think:

```
EXAMPLE / FLOW
Candidate Keys
    
Possible choices
    
Choose one
    
Primary Key
```

So:

> Every primary key is chosen from the available candidate keys.

NATURAL KEY

Simple Definition

A Natural Key comes from actual business data.

Examples:

```
EXAMPLE / FLOW
Email
EmployeeNumber
ISBN
```

EXAMPLE / FLOW

CountryCode
National identifier

A natural key already has business meaning.

Example:

EXAMPLE / FLOW

EmployeeNumber = EMP-10025

could uniquely identify an employee.

Natural Key Example

SQL

```
CREATE TABLE Employees
(
    EmployeeNumber VARCHAR(20) PRIMARY KEY,
    EmployeeName VARCHAR(100)
);
```

Here:

EXAMPLE / FLOW

EmployeeNumber

is both:

EXAMPLE / FLOW

Natural Key
+
Primary Key

Advantages of Natural Keys

They can:

- Have real business meaning
- Avoid an extra artificial identifier
- Sometimes be naturally unique

Problems with Natural Keys

Business data can change.

Suppose:

EXAMPLE / FLOW

Email

is the primary key.

Then the user changes their email.

Now a value used across many relationships must change.

Natural keys may also be:

EXAMPLE / FLOW

Long
Composite
Sensitive
Mutable

For these reasons, many application databases use surrogate primary keys.

SURROGATE KEY

Simple Definition

A Surrogate Key is an artificial identifier created only to identify a database row.

It normally has no business meaning.

Examples:

EXAMPLE / FLOW

CustomerId = 1001

OrderId = 500023

EmployeeId = 25

The number itself doesn't describe the customer, order, or employee.

Surrogate Key Example

SQL

```
CREATE TABLE Customers
(
    CustomerId BIGINT IDENTITY(1,1) PRIMARY KEY,

    Email VARCHAR(255)
        NOT NULL
        UNIQUE,

    CustomerName VARCHAR(100)
```

SQL

```
        NOT NULL
);
```

Here:

EXAMPLE / FLOW

CustomerId
=
Surrogate Key

while:

EXAMPLE / FLOW

Email

is a business/candidate key protected by UNIQUE.

Why Surrogate Keys Are Common

They are usually:

EXAMPLE / FLOW

Small
Stable
Simple
Easy to reference
Independent of business changes

If the customer's email changes:

EXAMPLE / FLOW

CustomerId = 100

doesn't need to change.

When to Use Surrogate Keys

Common for:

EXAMPLE / FLOW

Customers
Orders
Employees
Products
Payments

EXAMPLE / FLOW

Transactions

especially when business identifiers:

EXAMPLE / FLOW

Can change
Are long
Are composite
Contain sensitive information

When Not to Use a Surrogate Key Alone

Do not assume that adding:

EXAMPLE / FLOW

```
Id INT IDENTITY PRIMARY KEY
```

solves all uniqueness requirements.

Example:

EXAMPLE / FLOW

```
UserId   Email
--------------------
1        john@test.com
2        john@test.com
```

The surrogate primary key is valid, but the business data is wrong if emails must be unique.

You still need:

SQL

```
UNIQUE (Email)
```

This is extremely important.

Natural Key vs Surrogate Key

Natural KeySurrogate Key	
Business meaning	No business meaning
May change	Normally stable
Can be large	Usually small

Natural KeySurrogate Key	
May contain multiple columns	Usually one column
Example: Email	Example: CustomerId
Can expose business information	Internal technical identifier

A common modern design is:

EXAMPLE / FLOW

Surrogate Primary Key
+
Unique Business Constraint

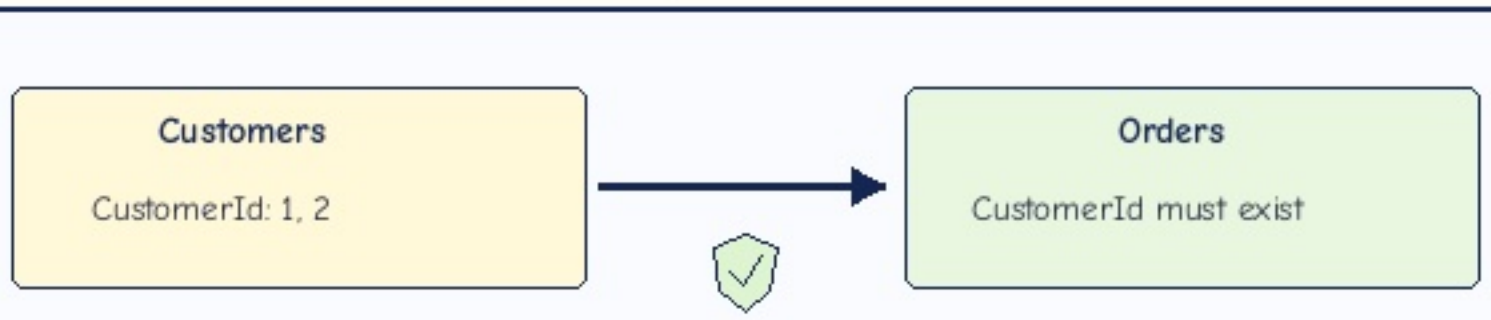
Example:

SQL

```
CustomerId BIGINT IDENTITY PRIMARY KEY,

Email VARCHAR(255) UNIQUE
```

REFERENTIAL INTEGRITY



Simple Definition

Referential Integrity means relationships between tables remain valid.

Example:

EXAMPLE / FLOW

Orders.CustomerId

must reference an existing:

EXAMPLE / FLOW

Customers.CustomerId

Foreign keys are the primary database mechanism used to enforce referential integrity.

Example

Customers

EXAMPLE / FLOW

```
CustomerId
----------
1
2
```

Orders

Valid:

EXAMPLE / FLOW

```
OrderId   CustomerId
-------------------
101       1
102       2
```

Invalid:

EXAMPLE / FLOW

```
OrderId   CustomerId
-------------------
103       999
```

because customer 999 doesn't exist.

Foreign key prevents it.

Referential Integrity and DELETE

Suppose:

EXAMPLE / FLOW

Customer 1

has orders.

Then someone tries:

SQL

```
DELETE FROM Customers
WHERE CustomerId = 1;
```

If a normal foreign key exists and dependent orders still reference customer 1, SQL Server prevents the delete.

This protects the relationship.

Referential Actions

SQL Server can define what happens when a referenced parent row is deleted or updated.

Common actions include:

EXAMPLE / FLOW

```
NO ACTION
CASCADE
SET NULL
SET DEFAULT
```

ON DELETE NO ACTION

This is the normal protective behavior.

Example:

SQL

```
FOREIGN KEY (CustomerId)
REFERENCES Customers(CustomerId)
ON DELETE NO ACTION
```

If orders exist, SQL Server prevents deleting the customer.

Use when child data should protect the parent.

ON DELETE CASCADE

Deleting the parent automatically deletes related child records.

Example:

SQL

```
FOREIGN KEY (OrderId)
REFERENCES Orders(OrderId)
ON DELETE CASCADE
```

If an order is deleted:

EXAMPLE / FLOW

```
Order
  ⌧
Related OrderItems also deleted
```

When CASCADE Is Useful

Good example:

EXAMPLE / FLOW

Order
 OrderItems

If an order is intentionally removed, its item rows may have no independent meaning.

When CASCADE Can Be Dangerous

Imagine:

EXAMPLE / FLOW

Customer
 Orders
 Payments
 ShipmentHistory

A cascade configured carelessly could remove a huge amount of important data.

Never use cascade simply because it is convenient.

Understand the business lifecycle first.

ON DELETE SET NULL

When the parent is deleted, the foreign key becomes:

EXAMPLE / FLOW

NULL

Example:

SQL

```
FOREIGN KEY (ManagerId)
REFERENCES Employees(EmployeeId)
ON DELETE SET NULL
```

If a manager row disappears:

EXAMPLE / FLOW

ManagerId
 NULL

The foreign key column must allow NULL.

Complete Practical Example

Let's combine these concepts.

SQL

```
CREATE DATABASE CompanyDb;
GO

USE CompanyDb;
GO

CREATE TABLE Departments
(
    DepartmentId INT
        IDENTITY(1,1)
        CONSTRAINT PK_Departments
        PRIMARY KEY,

    DepartmentName VARCHAR(100)
        NOT NULL,

    DepartmentCode VARCHAR(20)
        NOT NULL,

    CONSTRAINT UQ_Departments_Name
        UNIQUE (DepartmentName),

    CONSTRAINT UQ_Departments_Code
        UNIQUE (DepartmentCode)
);
GO
```

Now employees:

SQL

```
CREATE TABLE Employees
(
    EmployeeId BIGINT
        IDENTITY(1,1)
        CONSTRAINT PK_Employees
        PRIMARY KEY,

    EmployeeNumber VARCHAR(20)
        NOT NULL,

    FirstName VARCHAR(100)
        NOT NULL,

    LastName VARCHAR(100)
        NOT NULL,

    Email VARCHAR(255)
        NOT NULL,

    DepartmentId INT
```

SQL

```
    NOT NULL,

Salary DECIMAL(12,2)
    NOT NULL,

EmploymentStatus VARCHAR(20)
    NOT NULL
    CONSTRAINT DF_Employees_Status
    DEFAULT 'Active',

CreatedAt DATETIME2(3)
    NOT NULL
    CONSTRAINT DF_Employees_CreatedAt
    DEFAULT SYSUTCDATETIME(),

CONSTRAINT UQ_Employees_EmployeeNumber
    UNIQUE (EmployeeNumber),

CONSTRAINT UQ_Employees_Email
    UNIQUE (Email),

CONSTRAINT CK_Employees_Salary
    CHECK (Salary >= 0),

CONSTRAINT CK_Employees_Status
    CHECK
    (
        EmploymentStatus IN
        (
            'Active',
            'Inactive',
            'Terminated'
        )
    ),

    CONSTRAINT FK_Employees_Departments
        FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
GO
```

Understand the design:

EXAMPLE / FLOW

EmployeeId
- BIGINT
- IDENTITY
- Surrogate Primary Key

EmployeeNumber

EXAMPLE / FLOW

- Natural/Candidate Key
- UNIQUE

Email
- Candidate Key
- UNIQUE

FirstName
- Required
- NOT NULL

DepartmentId
- Foreign Key

Salary
- CHECK >= 0

EmploymentStatus
- DEFAULT Active
- CHECK approved values

CreatedAt
- Default UTC timestamp

What Happens Without These Constraints?

Suppose we had no constraints.

Then this could happen:

EXAMPLE / FLOW

```
EmployeeId       = duplicate
EmployeeNumber   = duplicate
Email            = duplicate
DepartmentId     = department that doesn't exist
Salary           = -50000
Status           = 'ABCDEFG'
FirstName        = NULL
```

Application code may try to prevent these values.

But database constraints guarantee that SQL Server itself protects the rules.

Constraint Performance Impact

Constraints provide integrity, but they are not free.

When inserting or updating data, SQL Server may need to check:

EXAMPLE / FLOW

Primary Key
Unique Constraint
Foreign Key
Check Constraint
NOT NULL

That creates some processing overhead.

However:

> Removing important constraints merely to make writes slightly faster is usually a bad design decision.

Correctness is normally more important than avoiding tiny validation costs.

For real performance issues, measure the workload before removing integrity rules.

Key Performance Considerations

Primary Key / UNIQUE

Usually supported by indexes.

Benefits:

EXAMPLE / FLOW

Fast uniqueness checking
Potential faster lookups

Cost:

EXAMPLE / FLOW

Index storage
INSERT/UPDATE maintenance

Foreign Key

Provides relational integrity.

SQL Server doesn't automatically create a child-side index simply because a foreign key exists.

Frequently used foreign key columns often benefit from indexing.

CHECK / NOT NULL

Usually inexpensive validation compared with the value they provide.

They can also give SQL Server useful knowledge about valid data.

Common Mistakes

Using IDENTITY as if it were the Primary Key

Wrong idea:

EXAMPLE / FLOW

IDENTITY = Primary Key

Correct:

EXAMPLE / FLOW

IDENTITY
=
Value generator

PRIMARY KEY
=
Uniqueness + row identifier

Using a Surrogate Key but Forgetting Business Uniqueness

Bad:

EXAMPLE / FLOW

UserId	Email
1	john@test.com
2	john@test.com

if business rules say email must be unique.

Use:

EXAMPLE / FLOW

PRIMARY KEY (UserId)

UNIQUE (Email)

Using NULL for Required Data

If every order must have:

EXAMPLE / FLOW

CustomerId

then define:

```sql
CustomerId INT NOT NULL
```

rather than allowing unknown relationships unnecessarily.

Using Fake Values Instead of NULL

Bad:

```
TerminationDate = '1900-01-01'
```

when employee isn't terminated.

If absence genuinely means "not applicable yet":

```
TerminationDate = NULL
```

is more meaningful.

Using CHECK as a Replacement for Reference Data

A CHECK like:

```sql
CHECK (Status IN ('A','B','C'))
```

can be excellent when the list is small and stable.

But if statuses have additional business data, descriptions, workflows, or change frequently, a related reference table may be better.

Using CASCADE DELETE Without Understanding It

Always ask:

> If I delete this parent, should all dependent business records really disappear?

If not, don't use cascade.

Constraint Naming Best Practice

Instead of allowing random system-generated names, use consistent names.

Examples:

EXAMPLE / FLOW

PK_Employees

FK_Employees_Departments

UQ_Employees_Email

CK_Employees_Salary

DF_Employees_Status

Easy pattern:

EXAMPLE / FLOW

PK = Primary Key
FK = Foreign Key
UQ = Unique
CK = Check
DF = Default

This makes errors, deployments, and maintenance easier to understand.

Quick Comparison Table

ConceptMain Purpose	
Primary Key	Uniquely identifies each row
Foreign Key	Creates/enforces table relationship
UNIQUE	Prevents duplicate business values
NOT NULL	Requires a value
NULL	Represents unknown/missing/not applicable
DEFAULT	Supplies automatic value
CHECK	Validates a condition
IDENTITY	Automatically generates numeric values
Composite Key	Key consisting of multiple columns
Candidate Key	Any valid possible unique key
Natural Key	Business-value key
Surrogate Key	Artificial technical key
Referential Integrity	Keeps table relationships valid

Easy Memory Rule

EXAMPLE / FLOW

PRIMARY KEY
 Who is this row?

EXAMPLE / FLOW

FOREIGN KEY
 Which parent does this row belong to?

EXAMPLE / FLOW

UNIQUE
 Don't allow duplicates.

EXAMPLE / FLOW

NOT NULL
 A value is required.

EXAMPLE / FLOW

NULL
 Value is unknown/missing/not applicable.

EXAMPLE / FLOW

DEFAULT
 Use this value when none is provided.

EXAMPLE / FLOW

CHECK
 Value must follow this rule.

EXAMPLE / FLOW

IDENTITY
 Generate the number automatically.

EXAMPLE / FLOW

COMPOSITE KEY
 Multiple columns together form the key.

EXAMPLE / FLOW

CANDIDATE KEY
 Possible unique identifier.

EXAMPLE / FLOW

NATURAL KEY
 Identifier comes from business data.

EXAMPLE / FLOW

SURROGATE KEY
 Artificial identifier created for the database.

EXAMPLE / FLOW

REFERENTIAL INTEGRITY
 Keep relationships valid.

Interview Questions

1. What is a primary key?

A primary key uniquely identifies each row in a table. It must contain unique, non-null values, and a table can have only one primary-key constraint.

2. Can a table have multiple primary keys?

No. A table can have only one primary key, but that primary key can contain multiple columns, forming a composite primary key.

3. Can a primary key contain NULL?

No. A primary key must identify every row, so its key columns cannot be NULL.

4. What is a foreign key?

A foreign key is a column or column set that references a key in another table and helps enforce relationships between the tables.

Example:

EXAMPLE / FLOW

Orders.CustomerId
 Customers.CustomerId

5. What is referential integrity?

Referential integrity means child-table references remain valid. A foreign key prevents a

child row from referencing a parent value that does not exist.

6. What is the difference between primary key and unique constraint?

Both enforce uniqueness, but the primary key is the table's main row identifier. A table has only one primary key but can have multiple unique constraints.

7. What is NOT NULL?

NOT NULL means the column must contain a value for every row. SQL Server rejects inserts or updates that try to store NULL in that column.

8. What is NULL?

NULL represents an unknown, missing, or not-applicable value. It is different from zero, an empty string, or false.

9. How do you check for NULL?

Use:

```sql
IS NULL
```

or:

```sql
IS NOT NULL
```

rather than = NULL.

10. What is a DEFAULT constraint?

A DEFAULT constraint automatically provides a value when an INSERT omits that column.

Example:

```sql
Status VARCHAR(20) DEFAULT 'Active'
```

11. What is a CHECK constraint?

A CHECK constraint validates values using a Boolean condition.

Example:

```sql
CHECK (Salary >= 0)
```

prevents negative salaries.

12. Is IDENTITY a data type?

No. IDENTITY is a column property that automatically generates numeric values. It is commonly combined with INT or BIGINT.

13. Does IDENTITY guarantee uniqueness?

Not by itself. IDENTITY generates values, while a primary key or unique constraint enforces uniqueness.

14. Does IDENTITY guarantee gap-free numbers?

No. Rollbacks, deletes, caching, and failed transactions can create gaps in identity values.

15. What is a composite key?

A composite key consists of two or more columns that together uniquely identify a row.

Example:

EXAMPLE / FLOW

```
StudentId + CourseId
```

in an enrollment table.

16. What is a candidate key?

A candidate key is any minimal column or set of columns capable of uniquely identifying a row. One candidate key is normally selected as the primary key.

17. What is a natural key?

A natural key comes from real business data, such as employee number, ISBN, or another naturally unique business value.

18. What is a surrogate key?

A surrogate key is an artificial identifier created specifically for database row identification, such as:

SQL

```
CustomerId BIGINT IDENTITY
```

It normally has no business meaning.

19. Natural key vs surrogate key — which is better?

Neither is universally better. Surrogate keys are often simpler and more stable, while natural keys preserve business meaning. A common design uses a surrogate primary key plus a UNIQUE constraint on the natural business key.

20. What is ON DELETE CASCADE?

ON DELETE CASCADE automatically deletes child rows when the referenced parent row is deleted.

It should only be used when child records truly have no independent business meaning.

21. Does SQL Server automatically index foreign keys?

No. A foreign key enforces relational integrity, but SQL Server does not automatically create a child-side index for it. Frequently queried foreign keys often benefit from indexing.

22. What happens if a parent row is deleted while child rows reference it?

With the default protective behavior, SQL Server rejects the delete. Other configured actions can include CASCADE, SET NULL, or SET DEFAULT.

23. Can a foreign key reference a UNIQUE column instead of a primary key?

Yes, provided the referenced column or column set is protected by an appropriate unique candidate key that SQL Server can reference.

24. Why should important validation exist in the database if the API already validates?

Because data can reach the database through multiple paths: APIs, imports, scripts, jobs, SSMS, and other services. Database constraints provide final integrity protection regardless of entry point.

25. What is a common modern key design?

A common design is:

EXAMPLE / FLOW

Surrogate Primary Key
+
Unique Business Key
+
Foreign Keys
+
Appropriate NOT NULL / CHECK / DEFAULT constraints

Example:

SQL

```
CustomerId BIGINT IDENTITY PRIMARY KEY,
Email VARCHAR(255) NOT NULL UNIQUE
```

This provides a stable technical identifier while still protecting business uniqueness.

Final Revision

EXAMPLE / FLOW

PRIMARY KEY
 Unique row identity

FOREIGN KEY
 Relationship

UNIQUE
 No duplicate value

NOT NULL
 Required value

NULL
 Unknown/missing value

DEFAULT
 Automatic fallback value

CHECK
 Validation rule

IDENTITY
 Auto-generated number

COMPOSITE KEY
 Multiple columns form key

CANDIDATE KEY
 Possible unique key

NATURAL KEY
 Business key

SURROGATE KEY
 Artificial technical key

REFERENTIAL INTEGRITY
 Valid relationships

The most important design idea to remember is:

Keys identify and relate data; constraints protect the rules of that data. A good SQL Server design normally uses both application validation and database constraints so invalid information cannot easily enter the system.